

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Descarga, creación y ejecución de imagen Docker del minisitio de nuevo
ingreso de ESCOM

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

18 de septiembre de 2026

Índice

1. Introducción	2
2. Conceptos teóricos	2
2.1. Imagen Docker	2
2.2. Contenedor	2
2.3. Dockerfile	2
2.4. Mapeo de puertos	2
3. Desarrollo	3
3.1. Descarga del sitio	3
3.2. Dockerfile de Apache	3
3.3. Dockerfile de Nginx	3
3.4. Construcción de las imágenes	4
3.5. Ejecución de los contenedores	5
4. Resultados	6
4.1. Verificación en el navegador	6
4.2. Resumen de resultados	7
5. Conclusiones	7
6. Referencias Bibliográficas	8

Índice de figuras

1.	Contenido del minisitio ya descargado, listo para servirse en ambas imágenes	3
2.	docker build de las dos imágenes a partir del mismo contexto sitio/	4
3.	docker run de ambos contenedores y verificación del mapeo de puertos con docker ps	5
4.	Minisitio servido por el contenedor de Apache en http://localhost:8080	6
5.	Minisitio servido por el contenedor de Nginx en http://localhost:8081, con el primero seguir abierto en la otra pestaña	6

Índice de tablas

1.	Diferencias entre los dos Dockerfile del mismo sitio	4
2.	Estado final de los dos contenedores	7

1. Introducción

Esta práctica retoma el minisito de nuevo ingreso de ESCOM, ya descargado como copia fiel de https://www.escom.ipn.mx/nuevoingreso27_1/ en una práctica anterior del curso, y lo usa como caso de estudio para publicar un sitio estático mediante Docker. El objetivo no es el sitio en sí, sino el mecanismo: empaquetar el mismo contenido dentro de dos imágenes distintas, una basada en Apache y otra en Nginx, y comprobar que ambos servidores lo sirven de forma idéntica desde el punto de vista del navegador.

Docker resuelve un problema concreto de despliegue: que un servicio corra igual en la máquina de desarrollo, en la de un compañero de equipo y en el servidor de producción, sin depender de qué versión de Apache o de Nginx tenga instalada cada máquina. Este reporte documenta el proceso completo, desde el contenido ya descargado hasta los dos contenedores corriendo en paralelo, con evidencia de cada comando y del resultado final en el navegador.

2. Conceptos teóricos

2.1. Imagen Docker

Una imagen es una plantilla de solo lectura que contiene un sistema de archivos mínimo y las instrucciones necesarias para ejecutar una aplicación: el sistema operativo base, las dependencias y los archivos propios del proyecto [1]. Se construye una sola vez a partir de un Dockerfile y, a partir de ahí, es inmutable: para cambiar algo hay que reconstruirla, no modificarla en caliente. Cada instrucción del Dockerfile agrega una capa a la imagen, y Docker reutiliza las capas que no cambiaron entre una construcción y la siguiente.

2.2. Contenedor

Un contenedor es una instancia en ejecución de una imagen, con una capa de escritura efímera encima de las capas de solo lectura de esa imagen [1]. A diferencia de una máquina virtual, no incluye su propio kernel: se aísla del sistema anfitrión y de otros contenedores mediante los *namespaces* y los *cgroups* del kernel de Linux, pero en tiempo de ejecución comparte ese kernel con el host. Esa diferencia es lo que hace que un contenedor arranque en fracciones de segundo en vez de en el tiempo que toma arrancar un sistema operativo completo. De una misma imagen pueden crearse tantos contenedores como se necesiten, cada uno con su propio estado y ciclo de vida.

2.3. Dockerfile

El Dockerfile es un archivo de texto plano con instrucciones secuenciales que describen cómo construir una imagen [2]. Instrucciones como **FROM**, **COPY**, **RUN** o **EXPOSE** se ejecutan en orden, y cada una produce una capa nueva sobre la anterior. Para un sitio completamente estático, como el de esta práctica, el Dockerfile puede ser mínimo: basta con partir de una imagen que ya trae el servidor web instalado y copiar los archivos del sitio a la carpeta que ese servidor sirve por defecto.

2.4. Mapeo de puertos

Un contenedor corre en su propia red virtual y, aunque su proceso interno escuche en un puerto, ese puerto no es alcanzable desde el host a menos que se publique explícitamente [3]. La bandera **-p host:contenedor** de **docker run** le indica a Docker que reenvíe el tráfico que llega a un puerto del host hacia el puerto correspondiente dentro del contenedor. Esto es lo que permite correr dos contenedores que internamente usan el mismo puerto 80, uno servido con Apache y otro con Nginx, y aun así distinguirlos desde el navegador mediante dos puertos distintos del host.

3. Desarrollo

3.1. Descarga del sitio

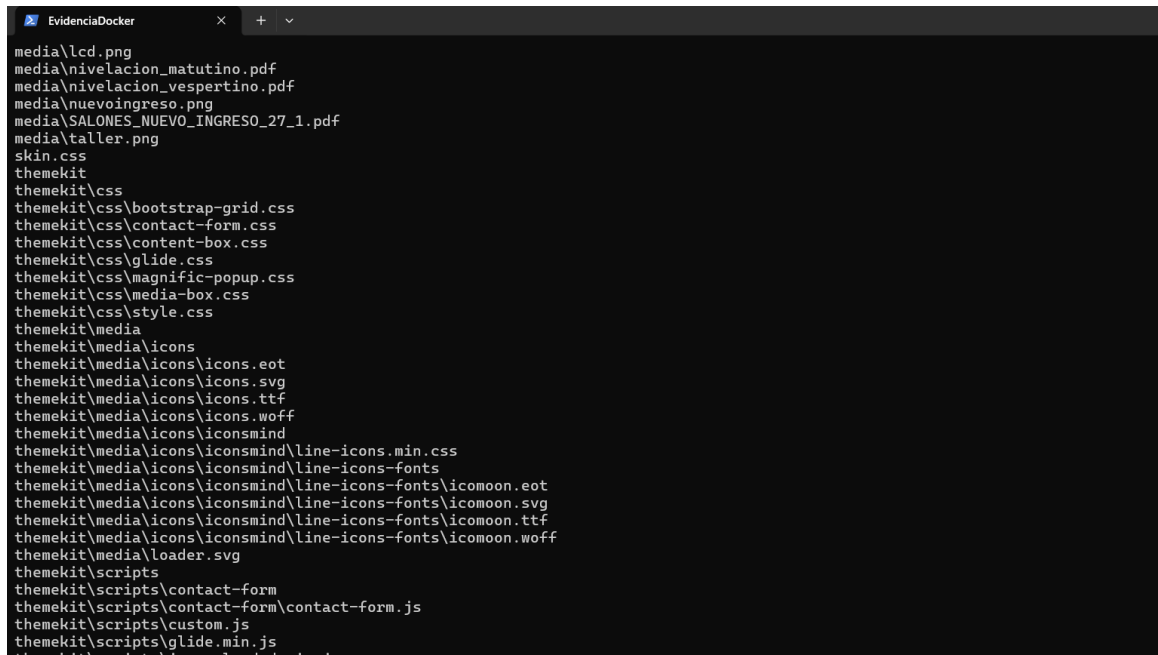


Figura 1: Contenido del minisitio ya descargado, listo para servirse en ambas imágenes

El contenido del minisitio ya estaba descargado de una práctica anterior del curso, que replicó https://www.escom.ipn.mx/nuevoingreso27_1/ archivo por archivo. La figura 1 muestra esa descarga: `index.html`, `skin.css` y la carpeta `themekit` con los estilos, íconos y scripts del tema, además de `media` con las imágenes y los PDF de asignación que el sitio original expone. Para esta práctica ese contenido se reutilizó como una única carpeta `sitio/`, compartida como contexto de construcción entre las dos imágenes, en vez de duplicar los mismos archivos dentro de cada una.

3.2. Dockerfile de Apache

Código 1: Dockerfile.apache

```
1 FROM httpd:2.4-alpine
2 COPY . /usr/local/apache2/htdocs/
3 EXPOSE 80
```

- `FROM httpd:2.4-alpine` parte de la imagen oficial de Apache httpd 2.4 sobre Alpine Linux [4], una base minimalista que reduce el tamaño final de la imagen frente a una distribución completa.
- `COPY . /usr/local/apache2/htdocs/` copia todo el contexto de construcción, es decir, el contenido de `sitio/`, al `DocumentRoot` que esa imagen sirve por defecto.
- `EXPOSE 80` documenta que el proceso de Apache dentro del contenedor escucha en el puerto 80. Por sí sola no publica nada hacia el host, eso lo hace `docker run -p`.

3.3. Dockerfile de Nginx

Código 2: Dockerfile.nginx

```
1 FROM nginx:alpine
2 COPY . /usr/share/nginx/html/
3 EXPOSE 80
```

La estructura es la misma que la del Dockerfile de Apache; solo cambian la imagen base y la carpeta de destino del COPY, que en la imagen oficial de Nginx es `/usr/share/nginx/html/` en vez de `/usr/local/apache2/htdocs/` [5]. La tabla 1 resume esa diferencia.

	Apache	Nginx
Imagen base	httpd:2.4-alpine	nginx:alpine
Carpeta de destino	/usr/local/apache2/htdocs/	/usr/share/nginx/html/
Puerto interno	80	80
Puerto publicado	8080	8081

Tabla 1: Diferencias entre los dos Dockerfile del mismo sitio

3.4. Construcción de las imágenes

```

--- docker build: imagen Apache ---
[+] Building 0.8s (7/7) FINISHED
=> [internal] load build definition from Dockerfile.apache
=> => transferring dockerfile: 532B
=> [internal] load metadata for docker.io/library/httpd:2.4-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 2.71kB
=> [1/2] FROM docker.io/library/httpd:2.4-alpine@sha256:3471d57791354e4eab0226c2f4f9a685ad5cab9e7ce64e2bb2bf7f9231231493
=> => resolve docker.io/library/httpd:2.4-alpine@sha256:3471d57791354e4eab0226c2f4f9a685ad5cab9e7ce64e2bb2bf7f9231231493
=> CACHED [2/2] COPY . /usr/local/apache2/htdocs/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:39348afecc576f79fe82ae699988af4a27be4fab9521d478c58da0001de09a9
=> => exporting config sha256:a25c7bfaa653e41f69ed26b843f39a3dc47707b09637336dde178eebb54f88ba
=> => exporting attestation manifest sha256:f82c5777c42112941566c8ff9696b9d82e76228c7e9df23f3fa32ac25fd3f879
=> => exporting manifest list sha256:2b9341a7453d8ac1c3493cd30acad81a011f047af45cfae88d46604faec3baf6
=> => naming to docker.io/library/minisitio-apache:latest
=> => unpacking to docker.io/library/minisitio-apache:latest

--- docker build: imagen Nginx ---
[+] Building 0.8s (7/7) FINISHED
=> [internal] load build definition from Dockerfile.nginx
=> => transferring dockerfile: 516B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 2.71kB
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
=> => resolve docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8
=> CACHED [2/2] COPY . /usr/share/nginx/html/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:154b32c5f02c5724b1b6714b5a7202b34b72326c44032b3c3f5e91d48104b

```

Figura 2: docker build de las dos imágenes a partir del mismo contexto sitio/

Código 3: Comandos de construcción

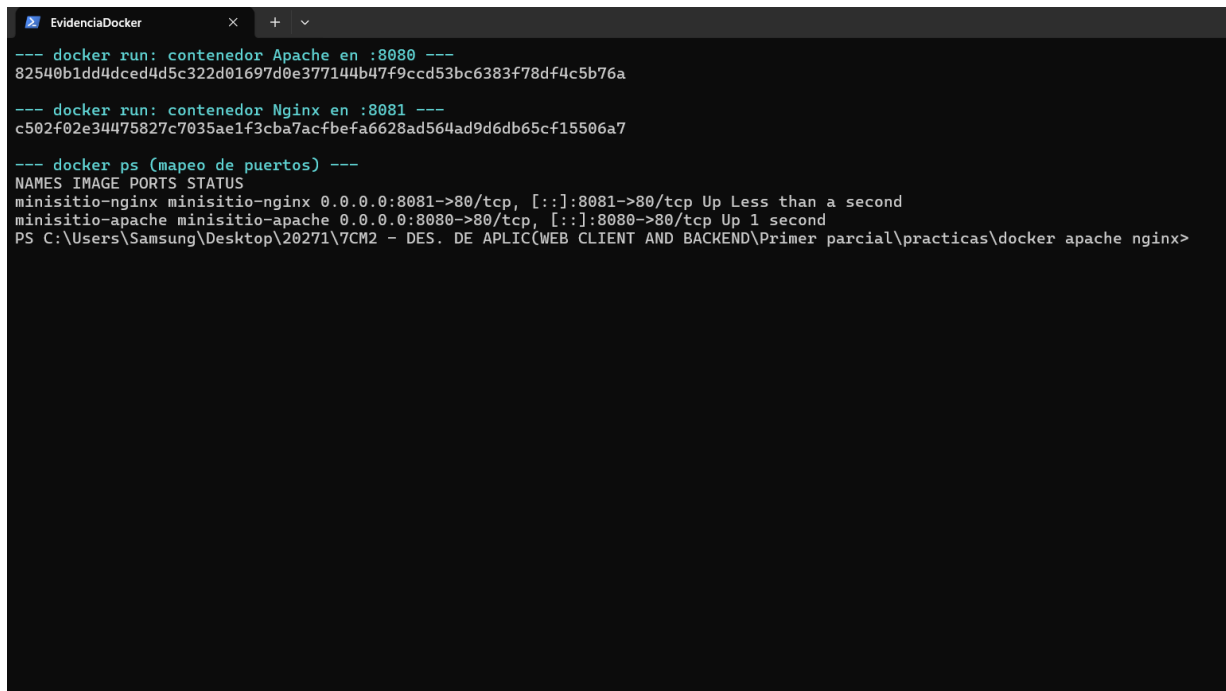
```

1 docker build -f Dockerfile.apache -t minisitio-apache ./sitio
2 docker build -f Dockerfile.nginx -t minisitio-nginx ./sitio

```

Como ninguno de los dos archivos se llama `Dockerfile` a secas, la bandera `-f` es obligatoria para indicar cuál usar en cada construcción. La bandera `-t` etiqueta la imagen resultante con un nombre legible, y `./sitio` es el contexto de construcción: la carpeta cuyo contenido se envía al daemon de Docker y queda disponible para las instrucciones `COPY` del Dockerfile. La figura 2 muestra las dos construcciones completas, cada una con sus siete pasos internos (carga del Dockerfile, resolución de la imagen base, transferencia del contexto, copia de archivos y exportación de la imagen final) terminando en `naming to docker.io/library/minisitio-apache:latest` y su equivalente para Nginx.

3.5. Ejecución de los contenedores



```
--- docker run: contenedor Apache en :8080 ---
82540b1dd4dced4d5c322d01697d0e377144b47f9ccd53bc6383f78df4c5b76a

--- docker run: contenedor Nginx en :8081 ---
c502f02e34475827c7035ae1f3cba7acfbefa6628ad564ad9d6db65cf15506a7

--- docker ps (mapeo de puertos) ---
NAME          IMAGE          PORTS          STATUS
minisitio-nginx minisitio-nginx 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp Up Less than a second
minisitio-apache minisitio-apache 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp Up 1 second
PS C:\Users\Samsung\Desktop\20271\7CH2 - DES. DE APLIC(WEB CLIENT AND BACKEND)\Primer parcial\practicadocker apache nginx>
```

Figura 3: docker run de ambos contenedores y verificación del mapeo de puertos con docker ps

Código 4: Comandos de ejecución

```
1 docker run --name minisitio-apache -dit -p 8080:80 minisitio-apache
2 docker run --name minisitio-nginx -dit -p 8081:80 minisitio-nginx
```

`-name` asigna un nombre legible al contenedor en vez de dejarlo con uno generado al azar. `-d` lo corre en segundo plano, `-i` mantiene abierta la entrada estándar y `-t` le asigna una pseudo-terminal; la combinación `-dit` es una convención común aunque, para un servidor web puramente estático como este, el contenedor no necesita interactividad para funcionar. La parte que sí es indispensable es `-p 8080:80` y `-p 8081:80`: publican el puerto 80 interno de cada contenedor en dos puertos distintos del host, para poder distinguir ambos servidores desde el navegador. La figura 3 confirma, con `docker ps`, que los dos contenedores quedaron arriba (*Up*) y con el mapeo de puertos esperado.

4. Resultados

4.1. Verificación en el navegador



Figura 4: Minisitio servido por el contenedor de Apache en <http://localhost:8080>



Figura 5: Minisitio servido por el contenedor de Nginx en <http://localhost:8081>, con el primero seguir abierto en la otra pestaña

Las figuras 4 y 5 muestran el mismo minisitio respondiendo en los dos puertos, con la barra de direcciones visible en cada caso. El contenido, los estilos y las imágenes se ven idénticos en ambas capturas porque provienen exactamente del mismo contexto de construcción; lo único que cambió fue el servidor web que lo entrega.

4.2. Resumen de resultados

Servidor	Imagen base	Puerto host	Respuesta HTTP
Apache httpd	httpd:2.4-alpine	8080	200 OK
Nginx	nginx:alpine	8081	200 OK

Tabla 2: Estado final de los dos contenedores

Ambos contenedores respondieron con código 200 al pedir la raíz del sitio, lo que confirma que el *DocumentRoot* quedó bien copiado en los dos casos y que el mapeo de puertos funciona en ambas direcciones.

5. Conclusiones

La práctica deja clara la diferencia entre imagen y contenedor de una forma que la definición por sí sola no transmite: la misma imagen `minisitio-apache` se pudo detener, eliminar y volver a correr con `docker run` sin reconstruirla, porque la imagen es la plantilla fija y el contenedor es la instancia desechable que se crea a partir de ella. También quedó claro que, para un sitio puramente estático, el Dockerfile no necesita más de tres instrucciones: la complejidad de un proyecto Docker está casi siempre en la aplicación que corre adentro, no en el mecanismo de empaquetado en sí.

El Dockerfile no dio ningún problema; lo complicado fue el entorno. El primer intento de `docker build` falló porque el daemon de Docker no estaba corriendo (*failed to connect to the docker API... check if the path is correct and if the daemon is running*), ya que Docker Desktop no se había iniciado en la sesión. Se resolvió iniciando Docker Desktop manualmente y esperando a que el motor terminara de levantar antes de reintentar. La otra decisión que tomó tiempo fue evitar duplicar dos veces los mismos catorce megabytes de recursos del sitio: en vez de copiar el contenido dentro de cada Dockerfile, se dejó una sola carpeta `sitio/` compartida como contexto de construcción entre ambas imágenes, usando la bandera `-f` para apuntar a cada Dockerfile por separado.

6. Referencias Bibliográficas

- [1] Docker Inc. *What is a Container?* 2024. URL: <https://www.docker.com/resources/what-container/> (visitado 18-09-2026).
- [2] Docker Inc. *Dockerfile reference*. 2024. URL: <https://docs.docker.com/reference/dockerfile/> (visitado 18-09-2026).
- [3] Docker Inc. *Networking overview: Published ports*. 2024. URL: <https://docs.docker.com/engine/network/#published-ports> (visitado 18-09-2026).
- [4] Apache Software Foundation. *httpd Docker Official Image*. 2024. URL: https://hub.docker.com/_/httpd (visitado 18-09-2026).
- [5] F5, Inc. *NGINX Docker Official Image*. 2024. URL: https://hub.docker.com/_/nginx (visitado 18-09-2026).